

Purkinje + PVJ + Regional Ionic Numerics

cardio-dance project

2026-05

Contents

1	Purkinje + PVJ + Regional Ionic Numerics	2
1.1	1. Domain decomposition	2
1.2	2. Sign and unit conventions	2
1.3	3. Myocardial monodomain (existing, recapped here)	3
1.4	4. Purkinje cable	3
1.4.1	4.1 Continuous form	3
1.4.2	4.2 Spatial discretization	3
1.4.3	4.3 Time stepping	4
1.4.4	4.4 Stability / dt selection	4
1.5	5. Purkinje ionic model (Stewart 2009)	4
1.5.1	5.1 Reference	4
1.5.2	5.2 Currents (target list)	4
1.5.3	5.3 State vector and integration	4
1.5.4	5.4 Status (2026-05): CellML codegen verified	5
1.5.5	5.5 Importing CellML reference code	6
1.6	6. Atrial ionic model (Grandi 2011)	6
1.7	7. Passive (leak) model	6
1.8	8. PVJ coupling	6
1.8.1	8.1 Continuous form	6
1.8.2	8.2 Sign convention	6
1.8.3	8.3 Mapping algorithm	7
1.8.4	8.4 Optional anatomical delay	7
1.8.5	8.5 Smeared injection (3D source-sink mismatch fix)	7
1.9	9. Regional ionic dispatcher	7
1.9.1	9.1 Mapping	7
1.9.2	9.2 State isolation (resolved 2026-05)	8
1.10	10. Verification matrix	8
1.11	11. Checkpoint format	9
1.11.1	11.1 Meta file (CHKPT_V3)	9
1.11.2	11.2 Shards	9
1.11.3	11.3 Format compatibility	9
1.11.4	11.4 Verified by	9
1.12	12. Change-log discipline	9
1.13	13. Fiber field (current limitation)	10
1.14	14. Half-ellipsoid LV demo	10

1.14.1	14.1 Geometry	10
1.14.2	14.2 Purkinje network	10
1.14.3	14.3 PVJ + smear	10
1.14.4	14.4 Pseudo-ECG probe	11
1.14.5	14.5 Run results (single rank, plain CG; full scale h=1mm)	11
1.15	15. Linear solver (Crank-Nicolson diffusion step)	11

1 Purkinje + PVJ + Regional Ionic Numerics

This document is the source-of-truth for the mathematical formulation behind the Purkinje stack. Every algorithm change must update this document in the same commit so reviewers can audit math/code drift in one diff.

1.1 1. Domain decomposition

Three coupled discrete systems share state every PDE step:

System	Discretization	Unknowns
Myocardium	3-D continuous Galerkin P1 on tetrahedra	V_m at true DOFs
Purkinje cable	1-D linear FE on a graph (subdivided edges)	V_p at FE nodes
PVJ coupling	Discrete gap-junction current at terminal $\leftrightarrow$ DOF	Per-link current I_{pvj}

Time stepping is operator-split Godunov with Crank-Nicolson on diffusion and Rush-Larsen + Forward Euler on reaction. Both PDEs march on the same outer Δt_{PDE} ; the cable substeps with $\Delta t_p \leq \Delta t_{\text{PDE}}$.

1.2 2. Sign and unit conventions

All currents in this codebase follow the TT06 / CellML convention:

$I_{ion} > 0$ is outward (repolarizing). $I_{stim} < 0$ is depolarizing.

Units (consistently throughout):

Quantity	Unit	Notes
Length	mm	mesh coordinates, fiber lengths
Time	ms	
Voltage	mV	Cell membrane potential
Concentration	mM (mmol/L)	$[Na^+]_i$, $[Ca^{2+}]_{ss}$, etc.
C_m	$\mu\text{F}/\text{mm}^2$	membrane capacitance per unit area
χ	1/mm	surface-to-volume ratio
σ	mS/mm	tissue conductivity
g_{ion}	mS/ μF	normalized cell conductance
I_{ion}, I_{stim}	$\mu\text{A}/\mu\text{F}$	normalized membrane current (dV/dt)

Verification: the monodomain RHS in `MonodomainStepper::BuildRhs` reads `-chi*Cm*M*(I_ion + I_stim + I_pvj)` — the negative sign and `chi*Cm` prefactor convert from “current per unit area” semantics to units consistent with the Crank-Nicolson system $AV^{n+1} = BV^n + \text{RHS}$.

1.3 3. Myocardial monodomain (existing, recapped here)

Continuous form on heart domain Ω_h :

$$\begin{aligned}\chi C_m \partial_t V_m &= \nabla \cdot (\sigma \nabla V_m) - \chi C_m (I_{ion}(V_m, \mathbf{s}) + I_{stim} + I_{pvj}^h), \\ \partial_t \mathbf{s} &= \mathbf{f}(V_m, \mathbf{s}).\end{aligned}$$

Spatial discretization: P1 CG, mass M and stiffness K assembled with the fiber-aware tensor $\sigma = \sigma_f \mathbf{f} \otimes \mathbf{f} + \sigma_s \mathbf{s} \otimes \mathbf{s} + \sigma_n \mathbf{n} \otimes \mathbf{n}$. With **AV-delay / fibrosis regions** the stiffness uses an attribute-wise multiplier $\alpha(x)$:

$$K_{ij} = \int_{\Omega_h} \alpha(x) (\nabla \phi_i)^T \sigma \nabla \phi_j \, dx,$$

where $\alpha = \alpha_{av_delay_sigma_scale}$ (0.02) on AV-delay attributes, $\alpha_{fibrosis_sigma_scale}$ (0.1) on fibrosis attributes, 1.0 elsewhere.

Code mapping: `Assembler.cpp` builds `region_sigma_scale_` as a `PWConstCoefficient` indexed by mesh attribute, then wraps the `FiberTensorCoefficient` in `ScalarMatrixProductCoefficient`. Single `DiffusionIntegrator` consumes the product.

1.4 4. Purkinje cable

1.4.1 4.1 Continuous form

Each Purkinje fiber is modeled as a 1-D cable. With per-edge axial conductivity g_a (mS/mm) and uniform C_m^p ($\mu\text{F}/\text{mm}^2$),

$$C_m^p \partial_t V_p = \partial_x (g_a \partial_x V_p) - I_{ion}^p(V_p, \mathbf{s}_p) - I_{pvj}^p - I_{stim}^p.$$

At branch points (graph vertices with degree > 2) currents are conserved by construction: the FE assembly’s nodal balance enforces Kirchhoff’s law because each incident edge contributes its own g_a/L_{seg} entry to the same row.

1.4.2 4.2 Spatial discretization

Each graph edge of length L is subdivided into k FE sub-segments of length L/k . Linear FE on each sub-segment yield the local stiffness

$$K_{loc}^e = \frac{g_a}{L/k} \begin{pmatrix} 1 & -1 \\ -1 & 1 \end{pmatrix}$$

and a lumped mass $M_{loc}^e = \frac{C_m^p L/k}{2} I$. Global K_p and $\text{diag}(M_p)$ are sparse-assembled in `PurkinjeCable-Solver::BuildSparseMatrices`.

Edge weight semantics: the third token in `.network` edge lines is the **axial conductance per mm of edge length**, i.e. g_a . If absent, the value defaults to `cfg.purkinje_edge_g_mS_per_mm`. The per-segment conductance contribution is then $g_{seg} = g_a/L_{seg}$.

1.4.3 4.3 Time stepping

Operator split per outer Δt_{PDE} :

1. Subdivide into $n_{\text{sub}} = \lceil \Delta t_{\text{PDE}} / \Delta t_p \rceil$ steps of $\Delta t = \Delta t_{\text{PDE}} / n_{\text{sub}}$.
2. Per substep:
 1. Compute $I_{\text{ion}}^p(V_p, \mathbf{s}_p)$ at current V_p .
 2. Advance $\mathbf{s}_p$ via Rush-Larsen (gates) + Forward Euler (concentrations) using the current V_p .
 3. Build external current $I_{\text{ext}} = I_{\text{pvj}}^p + I_{\text{stim}}^p$.
 4. Solve $(M_p + \frac{\Delta t}{2} K_p) V_p^{n+1} = (M_p - \frac{\Delta t}{2} K_p) V_p^n - \Delta t M_p (I_{\text{ion}}^p + I_{\text{ext}})$.

Linear solve: CG with Jacobi (mfem: :DSmooTher) preconditioner; the system matrices are cached and rebuilt only when Δt changes.

1.4.4 4.4 Stability / dt selection

Crank-Nicolson is unconditionally A-stable for the diffusion split. Reaction substep is governed by the fastest gating τ (I_Na: $\tau_m \approx 0.05$ ms near upstroke). Practical guidance: - $\Delta t_p \leq 0.01$ ms during upstroke (handled by Rush-Larsen exact gate update; FE on concentrations less sensitive) - $\Delta t_{\text{PDE}} / \Delta t_p \leq 4$ keeps PVJ delay quantization < 0.005 ms

1.5 5. Purkinje ionic model (Stewart 2009)

1.5.1 5.1 Reference

Stewart, P., Aslanidi, O. V., Noble, D., Noble, P. J., Boyett, M. R., & Zhang, H. (2009). *Mathematical models of the electrical action potential of Purkinje fibre cells*. Phil. Trans. R. Soc. A 367 (1896), 2225–2255. DOI 10.1098/rsta.2008.0283.

The Stewart model is itself a Purkinje variant of TT06 with HCN funny current I_f added and modified $I_{\text{to}}, I_{Ks}, I_{K1}$ kinetics.

1.5.2 5.2 Currents (target list)

$$I_{\text{ion}} = I_{\text{Na}} + I_{\text{CaL}} + I_{\text{to}} + I_{K_r} + I_{Ks} + I_{K1} + I_f^{Na} + I_f^K + I_{NaCa} + I_{NaK} + I_{pCa} + I_{pK} + I_{bNa} + I_{bCa} + I_{\text{sus}}$$

I_CaL (TT06/Stewart shifted GHK):

$$I_{\text{CaL}} = g_{\text{CaL}} df f_2 f_{\text{CaSs}} \frac{4(V - 15)F^2}{RT} \frac{0.25 [Ca]_{\text{SS}} e^{2(V-15)F/RT} - [Ca]_o}{e^{2(V-15)F/RT} - 1}.$$

Important: both the prefactor and the exponential argument use the shifted voltage $V_{\text{shift}} = V - 15$ mV. A historical bug in this codebase mixed $V - 15$ in the prefactor with V in the exponent and produced huge non-physiological CaL drive.

1.5.3 5.3 State vector and integration

20 states: [0]V, [1]m, [2]h, [3]j, [4]xr1, [5]xr2, [6]xs, [7]r, [8]s, [9]d, [10]f, [11]f2, [12]fCass, [13]y(I_f), [14]Cai, [15]CaSR, [16]CaSS, [17]Nai, [18]Ki, [19]Rprime.

Per substep: - States 1..13 (gates): Rush-Larsen $x \leftarrow x_{\infty} - (x_{\infty} - x) e^{-\Delta t/\tau}$. - States 14..19 (concentrations + RyR): Forward Euler $x \leftarrow x + \Delta t \dot{x}$, with positivity clamps on $[Ca^{2+}]_*$, $[Na^+]_i$, $[K^+]_i$.

1.5.4 5.4 Status (2026-05): CellML codegen verified

StewartPurkinjeModel now delegates to the **official CellML codegen** in `src/ode/stewart_generated.{h,c}`, lifted verbatim from `models.cellml.org/exposure/38cf8387b0707f0ef6947f009710aeb5/stewart_aslanidi_noble_noble_boyett_zhang_2009.cellml/@cellml_codegen/C`. Functions are prefixed `stewart_` to avoid linker collision with TT06's identical signatures.

1.5.4.1 Single-cell verification (test_stewart_single_cell) Brief (-52 $\mu A/\mu F \times 0.5$ ms) stim from MDP=-90 mV, free run for 1000 ms. Codegen output:

Quantity	Paper Fig. 2	Current code	Test bound
V_rest_pre	-91 mV (paced)	-74.2 (HCN-driven natural rest)	[-78, -68] ✓
V_peak	+30 mV (plateau)	+53.7 (single-stim spike)	[10, 60] ✓
V_plateau	+30 mV	+17.2 (10 ms post-stim)	[10, 40] ✓
APD90	340 ms	293 ms	[200, 400] ✓
V_at_min	-91 mV (paced)	-75.9 (post-AP rest)	< -70 ✓

The “natural” Stewart rest is **-74 mV**, not -91 mV: HCN/I_f drive keeps the cell perpetually rising slowly toward firing threshold (this is expected Purkinje autorhythmic behavior). The paper's -91 mV figures come from steady-state pacing where the cell has been clamped low between beats; a single-stim test from MDP=-90 instead settles to V_{rest}=-74 mV. We test against the codegen's actual single-stim morphology, not the steady-state pacing values.

1.5.4.2 Cable verification (test_purkinje_cable_cv) 51-node Stewart cable, 1 mm spacing (50 mm total), Crank-Nicolson + Rush-Larsen integration with `cfg.purkinje_edge_g_mS_per_mm = 0.05` (lumped edge axial conductance tuned for L_{seg}=1 mm, C_m=0.01 $\mu F/mm^2$ to give CV in the published Purkinje range). Cluster-stim of nodes 0,1,2 at -150 $\mu A/\mu F$ for 1 ms; activation measured by V > -40 mV at probe nodes 10 and 40 (30 mm separation).

Quantity	Published Purkinje	Current code	Test bound
CV	1.5–4 m/s	5.4 m/s	[1.5, 6.5] ✓

CV depends sensitively on the lumped edge conductance `g_a` and on the discretization spacing; the test's 5.4 m/s is at the high end of published values but well inside the physiologically plausible window. For a tighter target, lower `purkinje_edge_g_mS_per_mm` further (CV $\sim \sqrt{g_a/C_m}$).

1.5.4.3 Diagnostic tooling `tools/debug_stewart_currents` now drives the production model directly (no shadow-integration); it dumps `t,v,I_total` CSV at 0.05 ms intervals. Used to verify the codegen-driven AP shape visually after any future refactor that touches Stewart integration.

1.5.5 5.5 Importing CellML reference code

Long-term remediation: 1. Download Stewart 2009 from <https://models.physiomeproject.org/exposure/...> (CellML 1.0 XML). 2. Run OpenCOR or cellml-tools to emit C with initConsts, computeRates, computeVariables matching the TT06 generated code style. 3. Replace src/ode/StewartPurkinjeModel.cpp body with thin wrappers delegating to the generated functions, mirroring the tt06_generated.{h,c} pattern. Keep the gate/concentration index lists (kGateMaps) as the only hand-written part. 4. Re-enable the strict physiological asserts in test_stewart_single_cell.cpp and remove the WILL_FAIL marker.

1.6 6. Atrial ionic model (Grandi 2011)

Grandi2011Model is a **reduced 15-state compact port** retaining the dominant atrial currents: I_{Na} , $I_{CaL}^{\text{simplified}}$, I_{Kr} , I_{Ks} , $I_{to,f}$, I_{Kur} , I_{K1} , I_{NaK} , I_{NaCa} , I_{pCa} , I_{bNa} , I_{bCa} . The simplified I_{CaL} uses a constant driving force ($V - 60$) rather than full GHK, matching what is typical for “minimal-model” atrial sims when only conduction (not Ca dynamics) matters.

Status (2026-05): no single-cell verification test exists yet for Grandi. Use this model only for atrial-region conduction studies, not for APD/AERP analysis.

Reference (target): Grandi, E., Pandit, S. V., Voigt, N., Workman, A. J., Dobrev, D., Jalife, J., & Bers, D. M. (2011). *Human atrial action potential and Ca^{2+} model: sinus rhythm and chronic atrial fibrillation*. Circulation Research 109 (9), 1055–1066.

1.7 7. Passive (leak) model

$$I_{ion}(V) = g_{leak}(V - V_{rest}).$$

Used in AV-delay and fibrosis regions to suppress active depolarization while allowing electrotonic spread. State-free, so checkpoint is just the two parameters (g_{leak} , V_{rest}).

1.8 8. PVJ coupling

1.8.1 8.1 Continuous form

For each terminal Purkinje node t mapped to nearest myocardial true DOF $d(t)$:

$$I_{pvj,t} = g_{pvj} [V_p^t - V_m^{d(t)}] s_{pvj}.$$

- g_{pvj} in mS/ μ F (config: pvj_g_mS, default 0.5) - s_{pvj} unitless scale (config: pvj_current_scale, default 1.0)

1.8.2 8.2 Sign convention

When $V_p > V_m$ (Purkinje more depolarized), $I_{pvj} > 0$. The **myocardial RHS contribution** is $+I_{pvj}$ at DOF $d(t)$, treated with the same + sign as I_{ion} (outward semantics). So the actual contribution to the RHS is $-\chi C_m M(\dots + I_{pvj})$, i.e. when $V_p > V_m$ the negative sign **depolarizes** V_m . ✓

The **Purkinje cable** sees $-I_{pvj}$ at terminal t (Newton's third law: charge flows from Purkinje to myocardium, so Purkinje loses positive ion flux). Implementation in `PurkinjeCableSolver::Advance`: `i_pvj_p[fe] = pvj_g * scale * (vp[fe] - vm)`. Same sign as `BuildHeartCouplingCurrent` because both are added to their respective `I_ion` term.

1.8.3 8.3 Mapping algorithm

`PvjCoupler::BuildMapping`: 1. Each rank computes nearest local true DOF for each terminal. 2. `MPI_Allreduce(MPI_DOUBLE_INT, MPI_MINLOC)` selects the global owner. 3. Terminal-DOF pairs farther than `pvj_max_dist_mm` are dropped. 4. Each owner appends a `LocalPvjLink {purkinje_node, heart_tdof, dist}` to its `local_links_`.

Coordinate extraction uses MFEM `FunctionCoefficient` projected onto the P1 space, then `GetTrueDofs` — guarantees exact match with monodomain DOFs.

1.8.4 8.4 Optional anatomical delay

When `pvj_delay_ms > 0`: - After each Purkinje step, the current V_m at terminals is appended to a ring buffer (`PvjCoupler::AppendDelayedSample`). - The cable advance reads the sample with timestamp $\leq t_{\text{mid}} - \text{delay}$ (`ReadDelayedSample`) — implements a transport delay of `delay_ms` between heart and Purkinje sampling.

Buffer trim keeps history within $4 \times \text{delay}$ for safety.

1.8.5 8.5 Smeared injection (3D source-sink mismatch fix)

Under default single-DOF injection, the heart-side current is added to one true DOF nearest the terminal. On 3D unstructured tet meshes with cell size h comparable to or larger than the cardiac space constant $\lambda = \sqrt{D\tau_{\text{AP}}} \approx 0.3$ mm, this point source is drained by 8-12 surrounding resting DOFs *before* `I_Na` can establish locally — the activated patch is too small to self-sustain. Symptom: V_m briefly peaks near the injection then collapses back without propagation.

`pvj_smeared_radius_mm > 0` distributes the per-terminal current over a ball of radius R around the anchor DOF:

$$I_{pvj}^{(i)} = w_i g_{pvj} s_{pvj} (V_m^{(i)} - V_p) \quad \text{for all } i \in \mathcal{B}_R(\text{anchor})$$

with $w_i = 1/|\mathcal{B}_R|$ uniform; the per-terminal total injection magnitude is unchanged. Anchor DOF is still used (single owner rank) for V_m feedback to the cable, so the `MPI_Allreduce SUM` in `AdvancePurkinje` still recovers exactly one owner-side value per terminal even though the smeared list contains multiple links.

Tested on the half-ellipsoid demo: with `pvj_smeared_radius_mm=0` the wave collapses, with `pvj_smeared_radius_mm=4` (full-scale) or `=2` (half-scale) it propagates outward through the wall.

1.9 9. Regional ionic dispatcher

1.9.1 9.1 Mapping

Mesh element attributes drive region assignment:

Region	Cell model	Default config tag
Ventricle	TT06	ventricle_volume_attrs
Atria	Grandi 2011	atria_volume_attrs
AV-delay	Passive	av_delay_volume_attrs
Fibrosis	Passive	fibrosis_volume_attrs

For each owned true DOF, the region with the **highest priority** among incident elements wins (priority order: AV-delay > Fibrosis > Atria > Ventricle). This guards against a DOF on a region boundary getting an inappropriate active model.

1.9.2 9.2 State isolation (resolved 2026-05)

Each child model is constructed with `size = LocalDofCount(region)`, NOT the full DOF count. `RegionalIonicModel::ComputeIon` and `AdvanceStates` gather V_m into a region-local buffer using `region_indices[r]`, invoke the child on that buffer, and scatter the result back into the global `I_ion` vector. So a child's per-DOF state index is **region-local**, never indexed by global true DOF.

Empty regions still allocate a 1-DOF placeholder so the `unique_ptr` is constructible, but that placeholder is never invoked because `per_region_count[r] == 0` short-circuits the run path.

Tested in `tests/test_regional_isolation.cpp`: a 1-D 4-element mesh with ventricle on elements 0-1 (TT06) and AV-delay on elements 2-3 (Passive) verifies that (a) DOFs partition the `tdof` set, (b) Passive at V_{rest} yields exactly 0 `I_ion`, (c) repeated `AdvanceStates` with extreme ventricular V does not corrupt AV-delay output.

1.10 10. Verification matrix

Required tests (see `tests/`):

Test	Scope	Status
<code>test_tt06_single_cell</code>	TT06 AP shape (existing)	✓ pass
<code>test_stewart_single_cell</code>	Stewart codegen AP morphology	✓ pass
<code>test_purkinje_cable_smoke</code>	Cable diffusion correctness (passive)	✓ pass
<code>test_purkinje_cable_cv</code>	50 mm Stewart cable CV in [1.5, 6.5] m/s	✓ pass
<code>test_pvj_active_sign</code>	PVJ depolarizes myocardium when $V_p > V_m$	✓ pass
<code>test_regional_isolation</code>	Regional dispatch state isolation	✓ pass
<code>test_checkpoint_model_id</code>	CheckpointIO refuses cross-model load	✓ pass
<code>test_grandi2011_single_cell</code>	Atrial AP shape	TODO

End-to-end: - Full Niederer benchmark with regional + Purkinje (pending mesh tools). - Calibration sweep over g_{pvj} , σ_{AV} vs target activation times (script: `tools/calibrate_purkinje.py`, TODO).

1.11 11. Checkpoint format

1.11.1 11.1 Meta file (CHKPT_V3)

Plain-text, written only by rank 0:

```
CHKPT_V3
<world_size>
<step>
<t_ms>
<model_id>
```

`model_id` is `IIonicModel::ModelId()` of the model that wrote the checkpoint. `LoadLatest` refuses to proceed if the runtime model's `ModelId()` does not match (or, for legacy V2/V0 metas without `model_id`, if the runtime model is not TT06).

1.11.2 11.2 Shards

- `vm_rank<NNNNNN>.gf` — `mfem::ParGridFunction::Save` per rank.
- `cell_rank<NNNNNN>.bin` — `IIonicModel::SaveState` per rank. Binary layout is model-specific; cross-model loads are caught by the `model_id` check before any binary is read.

1.11.3 11.3 Format compatibility

Meta header	Status
CHKPT_V3	current, supported
CHKPT_V2 (legacy)	TT06-only legacy; refused otherwise single-rank TT06 only

1.11.4 11.4 Verified by

`tests/test_checkpoint_model_id.cpp`: writes a TT06 checkpoint, then asserts `PassiveModel LoadLatest` returns false and a fresh `TT06Model LoadLatest` returns true with matching `step/t_ms`.

1.12 12. Change-log discipline

Every commit touching algorithms in this stack must include a one-line note in the message body:

```
Math:    <equation/property changed>
Code:    <file/function changed>
Mapping: <how the verification stays sound>
```

If a verification test no longer passes after a change, mark it `WILL_FAIL` in the same commit and add a `TODO` line; do not silently relax the assertion.

1.13 13. Fiber field (current limitation)

The fiber tensor coefficient $\sigma = \sigma_f \mathbf{f}\mathbf{f} + \sigma_s \mathbf{s}\mathbf{s} + \sigma_n \mathbf{n}\mathbf{n}$ is fully wired (`Assembler::InitializeFiberCoefficients` reads three .gf files when `use_fiber_gf=1`). Conduction velocity along fiber vs transverse scales as $\sqrt{\sigma_f/\sigma_t} = \sqrt{0.1334/0.0176} \approx 2.75\times$ for the project default values.

However the .gf files we currently emit (via `tools/finalize_half_ellipsoid_mesh.cpp --emit-fibers`) are **constant vectors** $\mathbf{f} = (1, 0, 0)$, $\mathbf{s} = (0, 1, 0)$, $\mathbf{n} = (0, 0, 1)$ at every DOF. The conductivity tensor is therefore fixed Cartesian diagonal $\text{diag}(\sigma_f, \sigma_s, \sigma_n)$ — anisotropic along $\hat{x}$ but **not** following the natural Streeter helix of real left ventricles (-60° at endo rotating to $+60^\circ$ at epi).

What this is sufficient for: - PVJ sign / magnitude verification - PetsC ASM iteration-count benchmark - Pipeline correctness demo (mesh + Purkinje + ECG) - Order-of-magnitude QRS duration

What this is **not** sufficient for: - Realistic T-wave morphology (lacks transmural repolarization gradient driven by helical fibers) - Multi-lead ECG differences - Bundle-branch-block clinical case studies

Adding rule-based fiber generation (TODO): implement Bayer-Blake- Trayanova 2012 LDRBM via four Laplace-Dirichlet solves ($\phi_{\text{epi}}, \phi_{\text{endo}}, \phi_{\text{base}}, \phi_{\text{long}}$), construct local transmural / longitudinal / circumferential coordinate frames from $\nabla\phi$, then rotate the sheet vector by the Streeter helix angle $\alpha(d/W) = \alpha_{\text{endo}} + (d/W)(\alpha_{\text{epi}} - \alpha_{\text{endo}})$ across wall fraction d/W . ~200 lines. Not implemented.

1.14 14. Half-ellipsoid LV demo

A worked end-to-end pipeline lives in `config/half_ellipsoid_purkinje.options` and produces the figures / movies / pseudo-ECG under `output/half_ellipsoid_purkinje/`.

1.14.1 14.1 Geometry

Half-ellipsoid hollow LV (open at base $x = 0$): - Outer ellipsoid: $a = 35, b = 22, c = 22$ mm - Inner cavity: $a_{\text{in}} = 28, b_{\text{in}} = 15, c_{\text{in}} = 15$ mm - Wall thickness ranges ~ 7 mm equator, tapering at apex.

Mesh by `tools/half_ellipsoid.geo` (gmsh OpenCASCADE CSG $\rightarrow$ Frontal- Delaunay 2D + Delaunay 3D). At $h = 1.0$ mm: 21,707 nodes / 124,715 tets. `tools/finalize_half_ellipsoid_mesh.cpp` reclassifies boundary triangles into epi / endo / base by true face-centroid distance.

A previous Cartesian-carve mesh tool (`generate_half_ellipsoid_case`) exists for reference but produces stair-stepped boundaries; superseded.

1.14.2 14.2 Purkinje network

`tools/generate_purkinje_tree.py` (Costabal-style fractal): from a root near the apex along $-\hat{x}$, with 30° bifurcation angles, depth 9, bifurcation probability 0.85. Default at full scale: 192 graph nodes / 91 terminals. The `bbox` argument confines growth to within the cavity.

1.14.3 14.3 PVJ + smear

Activation comes from Purkinje only (no direct myocardial stim). Cluster pace at root nodes 0/1/2 ($\sim 150 \mu\text{A}/\mu\text{F}$ for 1 ms) drives the cable; PVJ maps each terminal to the nearest endocardial DOF (44/91 typically)

map within `pvj_max_dist_mm=5`). Heart-side injection is smeared over a 4 mm ball (full scale) or 2 mm (half scale) to clear the source-sink hurdle.

1.14.4 14.4 Pseudo-ECG probe

PseudoEcg evaluates $\phi(\vec{x}_p, t) = \frac{\sigma_i}{4\pi\sigma_b} \sum_e V_e \nabla V_m^{(e)} \cdot (\vec{x}_p - \vec{c}_e) / |\vec{x}_p - \vec{c}_e|^3$ at element centroids, MPI_Allreduce SUM, write one CSV column per probe. Default config places one lead at (17.5, 80, 0) mm.

1.14.5 14.5 Run results (single rank, plain CG; full scale h=1mm)

Quantity at $t = 100$ ms	Value
Wall (100 ms simulated)	5m52s
KSP iter / step (plain CG)	50 ± 6
KSP iter / step (PETSc ASM)	5 ± 1
V_m min / max / mean	-93.7 / +31.3 / -66.6 mV
Purkinje terminals mapped	44 / 91

Visualisation artifacts (committed to git for inspection): -output/half_ellipsoid_purkinje/vm_movie.mp4
 -output/half_ellipsoid_purkinje/vm_movie_clipped.mp4 -output/half_ellipsoid_purkinje/pseudo_ecg.
 -output/half_ellipsoid_purkinje/gallery/*.png (mesh wireframe + cross-sections)

1.15 15. Linear solver (Crank-Nicolson diffusion step)

The CN system $AV^{n+1} = \text{RHS}$ with $A = \chi C_m / \Delta t \cdot M + \frac{1}{2}K$ is symmetric positive- definite (M and K both SPD; positive linear combination preserves SPD).

Available backends in `LinearSolverFactory`, controlled by `SimulationConfig`:

Setting	Backend	Use when
<code>use_petsc=1</code> (default)	PETSc KSP, options from env PETSC_OPTIONS	Production runs (project default)
<code>use_hypre_boomeramg=1</code>	MFEM CG + HypreBoomerAMG	Large meshes, $\text{MPI} \geq 16$
<code>use_hypre_block_jacobi=1</code>	MFEM CG + HypreSmoother (11-Jacobi)	Cheap PC, comparable to plain CG
(none of the above)	MFEM CG without preconditioner	Diagnostic / small problems

Project-default config/petsc_asm.opts:

```
-mono_ksp_type cg
-mono_ksp_max_it 500
-mono_ksp_rtol 1e-8
-mono_ksp_norm_type unpreconditioned
-mono_pc_type asm
```

```
-mono_pc_asm_overlap 0
-mono_sub_pc_type icc
-mono_sub_pc_factor_levels 0
```

ICC(0) (Incomplete Cholesky, zero fill) exploits SPD structure; overlap 0 is the cheapest Schwarz variant. On the half-ellipsoid 124k-tet problem this gives mean **5 KSP iter/step** vs **50 iter/step** for plain CG (10× reduction); see docs/petsc_asm_vs_cg_iter_bench.md.

Activation:

```
export PETSC_OPTIONS="$(grep -v '^##\|^$' config/petsc_asm.opts | tr '\n' ' ')"
./build/monodomain --config config/half_ellipsoid_purkinje.options
```